Université de Strasbourg

2025-26

Structures de données et algorithmes II

Enoncé - Devoir de TP libre

Il est impératif de respecter scrupuleusement l'ensemble des contraintes imposées. Tout manquement, même mineur, sera pénalisé : un projet pourtant correct pourra se voir attribuer la note de 0/20 s'il ne satisfait pas strictement aux exigences demandées.

Cadre du problème : Complétion automatique de mots

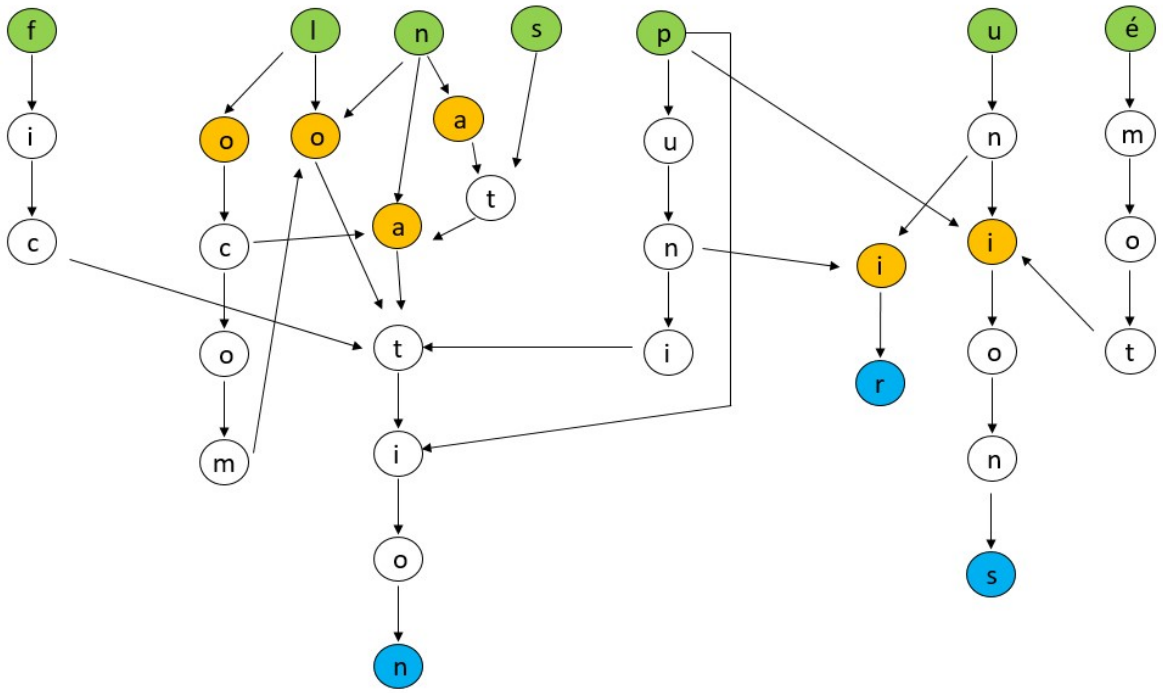


FIGURE 1 – Un exemple de graphe de mots (GDM) utilisé pour la complétion automatique. Ce graphe inclut les mots suivants : ["fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions", "unir", "punir", "station", "pion", "location"]. Il est composé de 34 nœuds et 39 arcs.7 nœuds sont des racines, 3 nœuds des feuilles, 24 sont internes.

Descriptif : Ce projet a pour objectif de concevoir et d'implémenter une structure de données permettant de représenter un lexique (**ensemble de mots)** sous la forme d'un **graphe orienté acyclique** que nous appellerons **graphe de mots (GDM)**. La Figure 1 montre un exemple de GDM. Dans notre représentation, **tout chemin partant d'un nœud racine** (nœud n'ayant aucun prédécesseur – en vert dans la figure 1) **et menant à une feuille** (nœud n'ayant aucun successeur – en bleu dans la figure 1) **correspond à un mot valide du lexique**. Cette approche permet de factoriser certaines sous-structures communes entre mots, tout en conservant une navigation efficace dans l'ensemble du vocabulaire.

Les nœuds du graphe sont stockés dans une **table de données** mettant en œuvre un partitionnement en fonction de l'alphabet et de l'état d'un nœud : *racine, feuille, isolé* ou *interne*. Pour chaque nœud, on stocke :

- la **liste des successeurs**, correspondant aux caractères possibles suivants ;
- la **liste des prédécesseurs**, utile pour certaines opérations de cheminement inverse, notamment partir d'une feuille pour remonter en sens inverse dans le graphe.
- un caractère
- un entier naturel utilisé comme « compteur ».

Chaque mot est analysé selon une décomposition en **préfixe, infixe et postfixe**, afin d'étudier les possibilités de partage de séquences à l'intérieur du graphe et d'améliorer la factorisation des chemins. Prenons l'exemple des mots unions et unir de la figure 1 : Ces mots sont décomposés en trois parties : « un - i - ons » et « un - i – r ». On voit sur la figure 1 que le préfixe « un » est commun aux deux mots. En fait c'est cette même factorisation que nous avions déjà exploitée au TD7 à l'aide d'un arbre n-aire. Ce qui change ici c'est la factorisation du postfixe : si nous prenons les mots lotion et natation décomposés en « l – o - tion » et « n – ata - tion », nous voyons que le postfixe « tion » est commun dans le graphe.

Une des caractéristiques de notre GDM est qu'avec la représentation que nous utilisons ici, seuls les préfixes et postfixes peuvent être factorisés. Le schéma de la figure 2 ci-dessous illustre un exemple de factorisation « incorrecte ». Il montre l'exemple des mots « relecture » et « élection » dans lequel l'infixe « lect » serait commun dans le graphe.

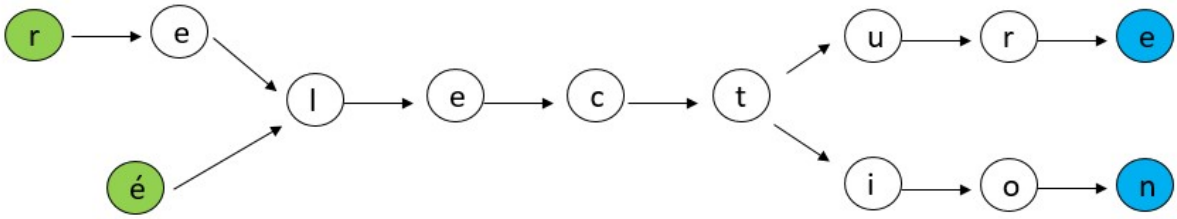


FIGURE 2 – Un exemple de graphe de mots (GDM) contenant: ["relecture", "élection"], mais produisant aussi des **mots invalides** : ["relection", "électure"].

Or on comprend aisément sur cet exemple que les mots non-existants « électure » et « relection » feraient alors également *implicitement* parti du lexique, car dans notre représentation tout mot est un chemin depuis une racine vers une feuille. Mais les mots électure et relection ne devraient pas exister dans le GDM. Le fait que l'infixe ne puisse pas être commun à plusieurs mots explique la duplication du « i » dans les mots « unions » et « unir ». Notez aussi que, pour cette raison, dans notre GDM, **un nœud peut avoir deux successeurs distincts qui contiennent pourtant le même caractère**. Ces nœuds sont marqués en orange dans la figure 1.

Toute la difficulté de ce projet consiste à bien structurer un mot en un préfixe – infixe – suffixe, de façon à éviter d'ajouter *par un effet de bord* des mots inexistants.

Finalement, la gestion des caractères reposera sur une **représentation partielle de l'UTF-8**, permettant de prendre en compte les accents et les caractères spéciaux.

1. Gestion des caractères UTF-8

L'UTF (Unicode Transformation Format) est un système d'encodage permettant de représenter l'ensemble des caractères utilisés dans les langues du monde au moyen de suites d'octets. Chaque caractère est d'abord associé à un point de code *Unicode* unique, puis ce point de code est converti en une séquence d'octets selon une variante d'UTF, la plus courante étant **UTF-8**. En UTF-8, que nous utiliserons pour ce projet, les caractères ASCII sont codés sur un seul octet, tandis que les caractères accentués ou spéciaux utilisent plusieurs octets (voir : https://fr.wikipedia.org/wiki/UTF-8).

Les seuls caractères que nous traitons dans le cadre de ce projet sont les suivants :

- Code sur un seul octet C0 , valide si dans l'intervalle : C0 ∈ [0x20, 0x7e]
- Code sur 2 octets (C0, C1), avec soit C0=0xc2 et C1∈ [0xa0, 0xbf], soit C0=0xc3 et C1∈ [0x80, 0xff]. Les codes avec C0=0xdf sont ignorés.
- Tous les autres codes sur trois octets (C0,C1,C2) ou quatre octets (C0,C1,C2,C3) sont aussi ignorés. Il s'agit des codes C0 == 0xe0 || C0 == 0xe2 || C0 == 0xed || C0 == 0xee pour trois octets et C0 == 0xf0 || C0 == 0xf3 || C0 == 0xf4 pour quatre octets.

Les codes « ignorés » seront considérés comme séparateurs (au même titre que le caractère espace ayant pour code C0=0x20). Bien qu'il existe des caractères étendus comme wchar_t ou char16_t, nous définissons dans le fichier include « base.h » un type caractère personnalisé `CarUTF8` codé sur deux octets et reprenant exactement le codage UTF-8. Ce type permet de représenter tous les caractères valides pour ce projet tels que précédemment définis.

1.1 Convertisseur de Chaine en CarUTF8

Sans utiliser de fonctions prédéfinie comme *mbtowc()* ou *mbrtoc16()* écrire une fonction qui transforme un flot de 4 octets maximum de type ***Car*** en un seul CarUTF8 valide :

```
/// // PRE: s!=NULL && u !=NULL && nc!=NULL
Bool enCarUTF8(Car s[4], CarUTF8* u, Nat* nc)
```

Le résultat est mis dans **u** et **nc**, où nc. indique le nombre d'octets effectivement utilisés dans s[4]. Le booléen renvoyé correspond à la validité du caractère.

Par exemple si s[4]={ 0x41 } (un seul octet) alors le booléen renvoyé vaut true et *nc=1 et *u= 0x0041. Si s[4]={ 0xc2, 0xab } (deux octets) alors le booléen renvoyé vaut true et *nc=2 et *u = 0xc2ab. Notez que l'on préserve exactement le codage UTF-8 initial. Si s[4]={ 0xdf, 0xbe } alors le booléen renvoyé vaut false et *nc=2 et *u =0x20. En effet, le caractère 0xdfbe, par ailleurs nommé *NKo Dantayalan*, est exclu et on le remplace par un séparateur ' '. Si s[4]={ 0xe0, 0xa0, 0x81 } alors le booléen renvoyé vaut faux, et *nc=3 et *u = 0x20, etc. Si s[0]=0 (aucun caractère) alors on renvoie true et *nc=1, *u=0.

Ecrire à présent une fonction qui convertit un flot arbitraire d'octets en un flot de CarUTF8. La fin du flot est marquée par un caractère nul (de valeur 0). Cette fonction utilise *enCarUTF8()*.

```
/// // PRE: s!=NULL && length!=NULL && n_invalides!=NULL
CarUTF8* enChaineUTF8(Chaine str,  Nat *length, Nat *n_invalides)
```

La fonction prend en entrée un flot de caractères **str** de longueur arbitraire (terminé par 0) et renvoie un flot de caractères CarUTF8 (terminé par 0x0000). On renvoie aussi dans **length** le nombre de caractères CarUTF8 du flot renvoyé et dans **n_invalides** le nombre de caractères invalides rencontrés dans **str**.

1.2 Autres opérations sur les chaines CarUTF8

Ecrire les fonctions usuelles sur les chaines de caractères, mais pour notre type personnalisé CarUTF8 :

```
/// // PRE: s!=NULL
Nat chaineUTF8Longueur(CarUTF8* s)
Bool estAlpha(CarUTF8 c)
Bool estNum(CarUTF8 c)
Ent compareCarUTF8(CarUTF8 x, CarUTF8 y)
```

La première fonction renvoie la longueur de la chaine (terminée par 0x0000). La fonction *estAlpha()* vérifie qu'il s'agit d'un caractère alphabétique. Nous considérons comme alphabétique tout caractère dans les intervalles [0x0041,0x005a] ou [0x0061,0x007a] ou [0xc380,0xc396] ou [0xc398,0xc3b6] ou [0xc3b9,0xc3bf]. Par exemples les caractères è, é, ê et ë de codes respectifs allant de 0xc3a8 à 0xc3ab sont alphabétiques. La fonction *estNum()* vérifie s'il s'agit d'un caractère numérique de 0 à 9 donc dans l'intervalle [0x0030,0x0039]. Le code 0xc2bc dont le caractère est '¼' ne sera notamment pas considéré comme numérique. Enfin la fonction *compareCarUTF8()* renvoie simplement la différence x - y de façon analogue à *strcmp*.

2. Les nœuds dans un Graphe de Mots (GDM)

2.1 Liste chaînée générique

Les nœuds du graphe de mots stockent les informations suivantes : le caractère CarUTF8, les listes des successeurs et prédécesseurs ainsi qu'un compteur. Nous avons besoin de définir un type liste pour les successeurs et prédécesseurs. Nous utilisons pour cela une liste chaînée (simple) générique `listeg`, ayant pour type de donnée void * (renommé en ***Jocker*** dans *base.h*) :

```
typedef struct s_maillong {
	Joker data; // la donnée générique
	struct s_maillong* succ; // pour le chainage
} *listeg;
```

Ecrire les opérations suivantes sur cette liste :

```
// OPERATIONS sur listeg
// création ex nihilo d'une liste vide
listeg nouvlg();
// destruction des maillons, mais pas des données
None detruirelg(listeg l);
// clonage des maillons de la liste, mais pas des données
listeg clonelg(listeg l);

// MODIFICATEURS
```

```
// PRE: x!=NULL
// adjonction d'un maillon en tête sans clonage de x
listeg adjtetelg(listeg l, Joker x);
// supprime le maillon de rang k, le premier étant k=0
// renvoie l si k>=longueur(l)
listeg supkiemelg(listeg l, Nat k);

// ACCES et SELECTEURS
// PRE: x!=NULL && comp!=NULL
// ne crée pas de nouvelle liste, mais
// renvoie un pointeur sur le premier maillon trouvé
// ou NULL si inexistant
listeg rechlg(listeg l, Joker x, fcomp comp);
// PRE: x!=NULL && comp!=NULL
// crée une nouvelle liste, contenant
// TOUS les éléments pour lesquels comp(x,l->data)==0
// ou NULL si inexistant
listeg rechalllg(listeg l, Joker x, fcomp comp);
// renvoie le maillon suivant ou NULL
listeg nextlg(listeg l);
// PRE: x!=NULL
// ne crée pas de nouvelle liste, mais
// renvoie un pointeur sur le maillon contenant exactement x
// ou NULL si inexistant
listeg elemlg(listeg l, Joker x);

// AUTRES OBSERVATEURS
// renvoie la donnée du premier maillon ou NULL
Joker tetelg(listeg l);
// renvoie vrai si liste n'a aucun maillon
Bool videlg(listeg l);
// nombre de maillons dans la chaine
Nat longueurlg(listeg l);
```

Notez que contrairement aux TD et TP vus jusqu'ici **nous ne clonons pas** les éléments de la liste, c'est pourquoi il n'est pas nécessaire d'ajouter des pointeurs de fonctions de clonage ou de destruction. Notez aussi que certaines opérations n'ont pas de précondition comme *tetelg()* qui renvoie alors NULL si la tête n'existe pas.

2.2 Noeud

Voici la structure de nœud que nous utilisons pour modéliser le graphe :

```
typedef struct snoeud {
	CarUTF8 car;  // UTF-8 avec 0xc2, 0xc3
	Nat count;     // pour compter les occurences
	listeg lsucc, lpred; // pour les arcs du graphe
} *Noeud;
```

Ecrire les opérations suivantes :

```
Noeud nouvNoeud(CarUTF8 c);
Nat hashNoeud(Noeud x);
Ent compareNoeudCar(Joker x, Joker y);
Nat degi(Noeud x);
Nat dege(Noeud x);
Bool estRacine(Noeud x);
Bool estFeuille(Noeud x);
Bool estIsole(Noeud x);
```

Pour la création d'un nouveau nœud, les listes des successeurs et prédécesseurs sont vides et le compteur est mis à 0. *hashNoeud()* est une fonction de hachage par extraction des 7 premiers bits du caractère CarUTF8. Ces bits correspondent à la valeur du code ASCII. Par exemple pour le caractère 0xc392 la fonction renvoie 0x12, pour 0x007a elle renvoie 0x7a. La valeur maximale de hachage est `N_ASCII`-1, avec `#define N_ASCII (0x80)-1a` définition est dans *base.h*. La fonction *compareNoeudCar()* utilise *compareCarUTF8* sur le caractère ***car*** du nœud pour comparer deux noeuds. Si les nœuds, même différents, contiennent le même caractère la fonction renvoie 0. *dege* et *degi* correspondent aux demi-degrés externe et interne du nœuds (nombre d'arcs ayant pour origine / extrémité ce nœud). Les autres fonctions vérifient le type de nœud.

3. Construction du graphe de mot

Le graphe de mot est modélisé par un ensemble de nœuds tels que définis précédemment, chaque nœud comprenant la liste des nœuds prédécesseurs et successeurs. Une table ou liste de nœuds suffit donc à représenter le graphe. C'est le principe des listes d'adjacence vu en cours. Pour augmenter l'efficacité des traitements, les nœuds du graphe sont stockés dans une **table de données** utilisant un partitionnement en fonction des caractères CarUTF8 et de l'état d'un nœud : *racine, feuille, isolé,* ou *interne*. Un nœud isolé est un nœud à la fois feuille et racine. Ces nœuds particuliers correspondent à des mots composés d'un seul caractère : par exemple la préposition 'à' ou le pronom adverbial 'y' en français. Un nœud interne est un nœud qui n'est ni feuille ni racine. Le partitionnement est mis en œuvre à travers quatre tableaux distincts. Chaque case du tableau est accédée par adressage calculé par extraction des 7 bits ASCII et une liste chaînée pour la gestion des collisions. Le graphe de mot (GDM) est défini par la structure suivante :

```
typedef struct sgdm {
       listeg racines[N_ASCII];
       listeg  feuilles[N_ASCII];
       listeg  isole[N_ASCII];
       listeg  interne[N_ASCII];
} GDM;
```

Un nœud **x** de type *racine* est inséré en tête de la liste chaînée de la case ***i*** du tableau `racines[N_ASCII]` où ***i*** est calculé par ***i=hashNoeud(x->car)***. De même pour les nœuds *feuilles*, *isolés* et *internes*.

3.1 Ecrire les opérations de base sur le graphe

```
None initGDM(GDM* g);
None detruireGDM(GDM* g);
```

qui sont les opérations de construction et destruction du GDM. La destruction doit libérer **tous les nœuds** et **tous les maillons des listes**.

3.2 Ecrire les constructeurs d'adjonction de nœud et d'arc

```
//PRE : g!=NULL && n!=NULL
//&&((est_racine&&!est_feuille)=>videlg(n->lpred)&&!existeNoeud(g,n->car,true,false))
//&&((est_feuille&&!est_racine)=>videlg(n->lsucc)&&!existeNoeud(g,n->car,false,true))
//&&((est_feuille&&est_racine)=>(videlg(n->lpred)&&videlg(n->lsucc))
//                                          && !existeNoeud(g,n->car,true,true))
None adjNoeud(GDM* g, Noeud n, Bool est_racine, Bool est_feuille);
```

```
//PRE : g!=NULL && nx!=NULL && ny!=NULL &&
// !estRacine(g,ny) && !estFeuille(g,nx) && !existeArc(g,nx,ny)
//
//POST: !existeCircuit(g)
None adjArc(GDM* g, Noeud nx, Noeud ny);
```

Observez les préconditions : on distingue l'adjonction de 4 types de noeuds selon qu'il s'agit d'une racine, d'une feuille, ou les deux à la fois (noeud isolé), ou aucun des deux (nœud interne). Le nœud est ajouté à l'un des quatre tableaux correspondant (un seul parmi les 4). Un nœud racine, feuille ou isolé est nécessairement **unique** dans le tableau : par exemple, il ne peut pas exister deux nœuds racine contenant le même caractère, c'est ce qu'implique la précondition *!existeNoeud(...)*. Par contre il peut exister plusieurs nœuds internes contenant le même caractère. Dans l'exemple de la figure 1, il n'existe qu'une seule racine contenant le caractère 'n', une seule feuille contenant 'n' et trois nœuds internes contenant 'n'.

Pour l'arc, il n'est pas possible d'avoir une origine qui est une feuille ou une extrémité qui est une racine. Un arc est aussi unique entre deux noeuds. Les nœuds ***nx*** et ***ny*** peuvent être des nœuds internes, donc ni racine, ni feuille. **Attention le graphe est acyclique : c'est-à-dire qu'après ajout de l'arc (post condition) le graphe ne doit contenir aucun circuit !**

3.3 Ecrire les observateurs suivants :

```
//PRE : g!=NULL
Noeud existeNoeud(GDM* g, CarUTF8 c, Bool est_racine, Bool est_feuille);
```

Si aucun noeud ayant pour caractère ***c*** existe alors la fonction renvoie NULL. S'il existe par exemple une racine contenant le caractère ***c***, celle-ci sera nécessairement unique (cf. précondition de l'adjonction d'un nœud). De même pour une feuille et un nœud isolé. Ces trois types de nœuds sont uniques pour un caractère donné. P**ar contre, il peut exister plusieurs nœuds internes contenant c**. Dans ce cas, on renvoie le premier rencontré dans la liste.

```
//PRE : g!=NULL
listeg tousNoeuds(GDM* g, CarUTF8 c, Bool est_racine, Bool est_feuille);
```

Comme précédemment. Mais renvoie une nouvelle liste de tous les nœuds contenant le caractère. Cette liste a pour longueur - au maximum - un pour les feuilles, racines et nœuds isolés.

```
//PRE : g!=NULL && nx!=NULL && ny!=NULL
Bool existeArc(GDM* g, Noeud nx, Noeud ny);
```

Vérifie l'existence d'un arc de ***nx*** à ***ny***. Rappel : le graphe est orienté.

```
//PRE : g!=NULL
Nat nbFeuilles(GDM* g);
Nat nbRacines(GDM* g);
Nat nbIsoles(GDM* g);
Nat nbInternes(GDM* g);
Nat nbArcs(GDM* g);
```

Ces opérations comptent respectivement le nombre de nœuds du graphe qui sont des feuilles, des racines, des nœuds isolés, des nœuds internes et le nombre total d'arcs.

3.4 Opérations sur les mots (chemins)

Ecrire une opération *existeMot()* qui vérifie l'existence d'un mot dans le GDM. Cette opération crée la liste des nœuds composant le mot, en partant d'une racine (tête de liste) et arrivant à une feuille (queue de liste). Si le mot n'existe pas dans le graphe on renvoie une liste vide. Cette fonction utilisera une fonction auxiliaire *existeChemin()* qui recherche un chemin d'un nœud ***nx*** vers un nœud ***ny*** par l'intermédiaire de ***nc*** **nœuds internes** ayant pour valeur les caractères d'une sous-chaine ***m***. Si nc==0 on vérifie simplement l'existence d'un arc de ***nx*** à ***ny***. Cette fonction renvoie une liste de nœuds telle que le maillon de tête contient ***nx*** et celui de queue ***ny***. Ou une liste vide s'il n'existe aucun chemin.

```
//PRE : g!=NULL && s!=NULL && nx!=NULL && ny!=NULL
// tous les noeuds de la liste renvoyée sont internes
// sauf éventuellement la tête et la queue
listeg existeChemin(GDM* g, CarUTF8* m, Nat nc, Noeud nx, Noeud ny);
 //PRE : g!=NULL && s!=NULL
listeg existeMot(GDM* g, CarUTF8* s);
```

Ecrire une opération *adjMot()* qui ajoute un mot dans le GDM. L'ajout doit se faire exactement de la manière décrite en annexe B. Il est conseillé de définir des fonctions auxiliaires qui déterminent le postfixe depuis la feuille et le préfixe depuis la racine. A chaque ajout de mot, les compteurs des nœuds concernés sont incrémentés de un. Même si le mot est ajouté une seconde fois et qu'aucun nouveau nœud n'est créé, les compteurs sont incrémentés. Pour le tout premier mot ajouté, tous les compteurs des nœuds vaudront 1. Ensuite, si le même mot est ajouté une seconde fois tous les compteurs vaudront deux.

```
//PRE : g!=NULL && s!=NULL
listeg adjMot(GDM* g, CarUTF8* s);
```

Attention à bien éviter d'ajouter des mots non valides par effets de bord. Traiter également le cas d'un mot composé d'une seule lettre. Enfin, les racines ont toutes des caractères distincts. De même pour les feuilles et les nœuds isolés.

3.5 Complétion automatique

Nous souhaitons utiliser le GDM à des fins de complétion automatique. Pour cela, écrire une fonction qui à partir d'une chaîne ***s*** renvoie une liste de tous les mots ayant pour préfixe s.

```
//PRE : g!=NULL && s!=NULL && mots!=NULL && max>0
Nat completion(GDM* g, CarUTF8* s, listeg mots[], Nat max);
```

Cette fonction renvoie le nombre de mots trouvés et les place sous forme de listes chaînées de noeuds dans le tableau *mots[ ]*. *max* représente la taille de ce tableau. Si le nombre de mots est supérieur à *max* on ne conservera que les *max* premiers mots. Aucun ordre alphabétique n'est demandé. Par exemple, en reprenant le lexique de la figure 1, si s=« na », alors *completion()* renvoie 2 et place dans le tableau *mots []* les chaines { « natation », « nation » }.

Annexe A : CONSIGNES

Le TP libre est à déposer – au plus tard – le **dimanche 3 mai 2026, 23h59 sur moodle**. Le fichier déposé est un unique fichier de nom ***gdm.c*** contenant l'intégralité du code source commenté, en respectant exactement le modèle du fichier ***canevas.c*** qui vous est fourni (pas de header séparé, un seul fichier, tous les types de données et prototypes des fonctions seront placés dans cet unique fichier *gdm.c*).

Le projet est à réaliser seul.

La note est fonction de la quantité et qualité ***du travail personnel*** réalisé.

Consignes spécifiques concernant le code source :

- Le programme doit être écrit en C, pas en C++. Aucune syntaxe C++ n'est tolérée.
- Il est interdit de faire d' « include » autre que « base.h ».
- Il est interdit de modifier « base.h ».
- Il est interdit d'utiliser directement les fonctions d'allocation mémoire du système. Il faut obligatoirement utiliser les macros MALLOC, MALLOCN, CALLOCN, REALLOC et FREE définies dans le fichier « base.h ».
- Il est interdit d'utiliser les types prédéfinis : *int, unsigned, float, char, union, enum*, etc. du langage C. Il faut obligatoirement **utiliser les seuls types** définis dans « base.h », c'est-à-dire : Bool, Nat, Reel, Car, Chaine, etc.
- Il est interdit de définir d'autres structures ou types de données que ceux fournis dans *canevas.c et base.h* : *CarUTF8, listeg, Noeud, GDM*. Il est aussi interdit d'ajouter à ces structures d'autres champs ou de modifier les *typedef* de quelque manière que ce soit. Si on souhaite définir d'autres types (comme liste, pile ou file, il faut faire une *implémentation directe* sans définir de type spécifique).
- Il est interdit de modifier le prototype des fonctions : il faut conserver exactement les prototypes tels que définis dans *canevas.c*.
- Il est autorisé d'ajouter des opérations ou fonctions auxiliaires.
- Aucune fonction ou variable « externe » n'est autorisée. Donc **pas de *printf*** - dans aucune fonction (sauf dans le *main*), pas de variables globales, pas d'appel à *qsort* non plus ou tout autre fonction du système Unix. Pour les opérations sur les chaînes de caractères, utiliser les macros de « base.h ».
- Les noms de fonctions ET de variables suivants sont interdits : *new, delete, volatile, static, mutable, register, namespace, using, asm, inline, template, typename, operator, explicit, constexpr, decltype, concept, requires, class, protected, public, private, explicit, friend, virtual, override, final, try, catch, throw*. Ce sont des mots réservés du langage C++ qui perturbent la compilation.
- Il est interdit d'utiliser des directives de précompilation # et ## : donc pas de *#undef, #if, #elif, #else, #endif, #ifdef, #ifndef, #pragma, #error, #warning*.
- L'opérateur de séquencement « virgule » est interdit même lorsqu'il s'agit d'initialiser des variables : `Nat a=0, b=1, c=2 ;` ou `for (i=0,j=5 ; i<5 ; i++,j--)`. Il faudra écrire par exemple : `Nat a=0 ; Nat b=1 ; Nat c=2 ;`
- Les opérateurs ternaire « ? : » et unaires « ++ » et « -- » sont autorisés.
- Les structures de « sauts » suivantes sont interdites : *break, continue, goto, switch, case*. Seul le *return* est autorisé.
- Pour définir des constantes, on utilisera *#define*. Par exemple : `#define omega 0 ;`
- Il est interdit d'utiliser *#define* pour définir autre chose que des **constantes** numériques. Donc *#define* ne doit pas être détourné pour définir des fonctions. Vous devez toujours définir des « vraies » fonctions (auxiliaires), **jamais des macros**. Les seules macros autorisées sont celles de « base.h ». Donc par exemple : #define max(a,b) ((a)>(b) ?(a):(b)) est interdit !

- Le cast (conversion de type) utilisera la syntaxe du C et non celle du C++ : par exemple : `Nat i = (Nat)x ;` et non pas : `Nat i = Nat(x) ;`
- Les commentaires // et /* */ sont autorisés et fortement recommandés !

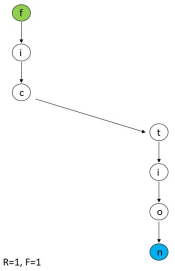
Enfin :

- Le code doit compiler sur Turing avec gcc sans erreur, ni warning. Attention ***nous utilisons systématiquement en plus de –Wall –Wextra les options : -Werror=vla et –Wc++-compat*** qui ont pour effets respectifs d'interdire les ***VLA (Variable Length Arrays)*** et certaines conversions implicites notamment concernant *void* ***.**
- Les fuites mémoire seront vérifiées et donneront, le cas échéant, lieu à une pénalité.

Annexe B : CONSTRUCTION DU GRAPHE

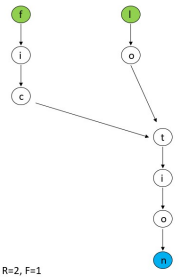
Nous montrons ici une à une les étapes de construction du graphe donné à titre d'exemple dans la figure 1 et dans le *main()*. Votre code doit reproduire exactement ce graphe à chaque étape. Les nœuds et les graphes doivent être **rigoureusement identiques** pour chaque adjonction de mot.

{ "fiction« };



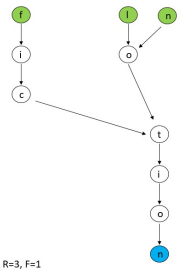
N=5, R=1, F=1
A=6

{ "fiction", "lotion" };



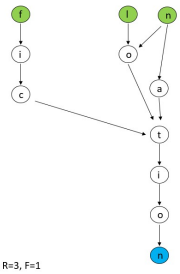
N=6, R=2, F=1
A=8

{ "fiction", "lotion", "notion" };



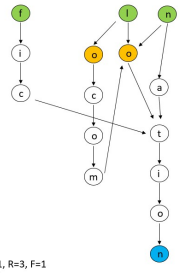
N=6, R=3, F=1
A=9

{ "fiction", "lotion", "notion", "nation" };



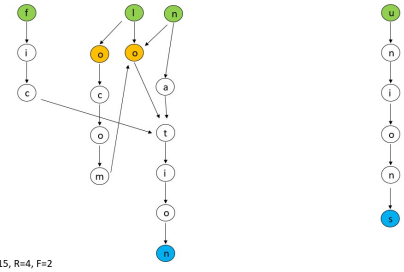
N=7, R=3, F=1
A=11

{ "fiction", "lotion", "notion", "nation", "locomotion" };



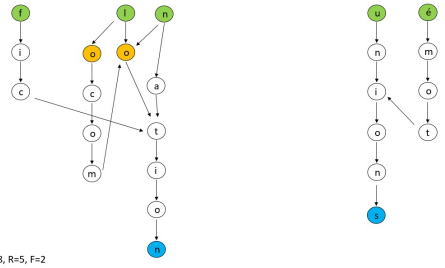
N=11, R=3, F=1
A=16

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions" };



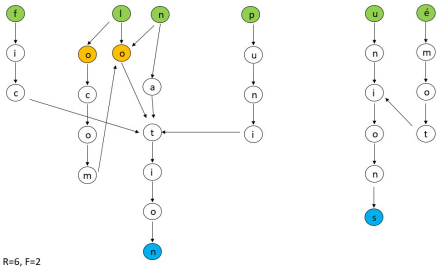
N=15, R=4, F=2
A=21

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions" };



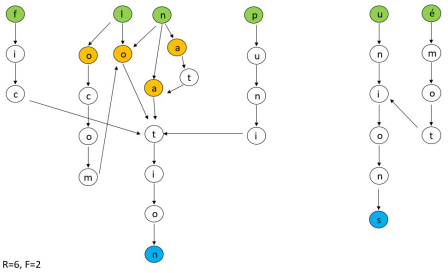
N=18, R=5, F=2
A=25

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition" };



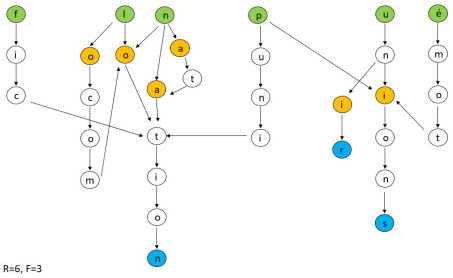
N=21, R=6, F=2
A=29

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation" };



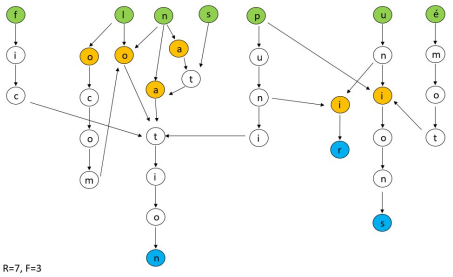
N=23, R=6, F=2
A=32

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions", "unir" };



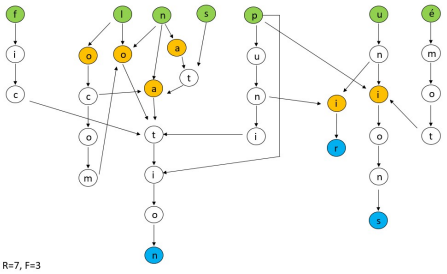
N=24, R=6, F=3
A=35

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions", "unir", "punir", "station" };



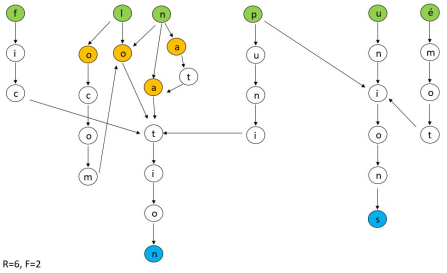
N=24, R=7, F=3
A=37

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions", "unir", "punir", "station", "pion", "location" };



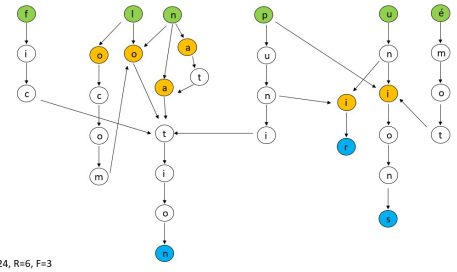
N=24, R=7, F=3
A=39

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions" };



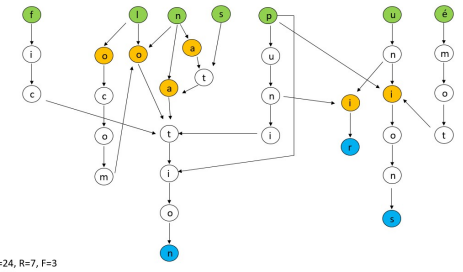
N=23, R=6, F=2
A=33

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions", "unir", "punir" };



N=24, R=6, F=3
A=36

{ "fiction", "lotion", "notion", "nation", "locomotion", "unions", "émotions", "punition", "natation", "pions", "unir", "punir", "station", "pion"};



N=24, R=7, F=3
A=38